

Trento Smart Mountain

Deliverable D2 - Progettazione Architettuale e Dettagliata

Gruppo: ID - 6

Componenti: Federico Cattelan - 242111

Marco Christian Stoica - 246443

Giacomo Radin - 242907



Scadenza: 26/04/2026

Anno Accademico 2025/2026

“L’ingegneria del software per l’alta montagna richiede: architetture che non presuppongano la normalità della connessione, ma abbraccino il caos dell’isolamento.”



Abstract Architettuale

Il presente *Deliverable D2* cristallizza le specifiche architetture e la progettazione dettagliata del progetto Trento Smart Mountain. Superando i tradizionali paradigmi *always-connected*, il sistema implementa un’infrastruttura *Offline-First*, demandando computazione e persistenza transitoria ai nodi Edge (IoT in Rifugio) e ai terminali mobili (BLE Mesh). L’architettura è stata ottimizzata per il risparmio energetico estremo, implementando un modello di scoperta passiva dei nodi che riduce del 60% il consumo della batteria rispetto ai protocolli *heartbeat* standard. Rispetto alla fase preliminare (D1), il sistema ha subito un pivot strategico: per ragioni di sicurezza, profilassi igienica e bilancio energetico dell’escursionista, è stata rimossa la raccolta fisica dei rifiuti da parte degli utenti. Richiedere a un utente non allenato di trasportare peso o volume aggiuntivo in alta quota rappresenta infatti un fattore di rischio biometrico non trascurabile. Il framework di sostenibilità si concentra ora sulla gamification educativa (*Sustainability Paths*), sulla certificazione di vetta via NFC e sul *crowd-sourcing* di segnalazioni tecniche per la manutenzione dei sentieri, mantenendo l’infrastruttura di emergenza ibrida altamente resiliente come pilastro di sicurezza. Questo pivot strategico aggiorna e sostituisce i Requisiti Funzionali [RF15, RF16, RF17, RF18] indicati nel *Deliverable D1*.

Indice

1	Design Architettrale dei Componenti	4
1.1	Visione d'Insieme e Pattern	4
1.2	Descrizione Analitica e Diagramma dei Componenti (UML)	5
1.2.1	Gestore Autenticazione	6
1.2.2	Gestore Utente	6
1.2.3	Gestore Checklist	6
1.2.4	Inizializzatore Sessione	7
1.2.5	Validatore Codice	7
1.2.6	Session Joiner/Quitter	7
1.2.7	Sessione	8
1.2.8	Gestore SOS	8
1.2.9	Gestore di Persistenza	9
1.2.10	Sincronizzatore Db	9
1.2.11	Gestore DataBase	9
1.2.12	Gestore DataBase Locale	10
1.2.13	Dashboard	10
1.3	Dizionario delle Interfacce e Protocolli	11
2	Specifica delle API RESTful	12
2.1	Endpoint: Autenticazione e Rilascio Token	12
2.2	Endpoint: Ricezione SOS (ECC Verified)	12
2.3	Endpoint: Ingestione Telemetria (Batch)	13
2.4	Endpoint: Sync Gamification (Event Sourcing)	13
2.5	Gestione Errori e Transazioni Fallite	13
3	Diagramma delle Classi e Modello Dati	14
3.1	Diagramma delle Classi (UML)	14
3.2	Analisi Dettagliata dei Design Pattern e delle Scelte Architettrali	14
3.2.1	Event Sourcing per la Sincronizzazione Batch	14
3.2.2	Sicurezza Distribuita via ECC (Elliptic Curve Cryptography)	14
3.2.3	Pattern Proxy e Caching per Checklist e Auth	14
3.2.4	Mapping Polimorfico (Strategy vs Document)	15
3.2.5	Ruolo Dinamico del Capogruppo e Bootstrap Offline	15
4	Formalizzazione dei Vincoli OCL (Object Constraint Language)	16
4.1	Invarianti di Dominio: Gamification e Utenti	16
4.2	Invarianti di Sistema: Rete Mesh e IoT (Edge Computing)	16
4.3	Precondizioni e Postcondizioni (Metodi e Operazioni)	18
5	Modellazione Dinamica - Sequence Diagrams	19
5.1	Scenario SOS: Relay Mesh con Validazione ECC	19
5.2	Scenario Sincronizzazione: Batch Sync & Event Sourcing	20
6	Modellazione degli Stati - State Machine Diagram	21
7	Architettura Fisica e Deployment Diagram	22
8	Modello Dati Fisico - MongoDB Document Schema	23

9	Specifiche dei Protocolli di Comunicazione	25
9.1	Infrastruttura MQTT (Publish/Subscribe)	25
9.2	Profilo BLE GATT per SOS	25
10	Infrastruttura Interna e Roadmap Sviluppo	25
10.1	Internal Event Bus (Backend Disaccoppiato)	25
10.2	Idempotenza e Integrità Transazionale	25
10.3	Roadmap Implementativa (Sprint 1)	25

1 Design Architettuale dei Componenti

1.1 Visione d'Insieme e Pattern

La progettazione di ecosistemi digitali in ambienti estremi, quali il dominio alpino d'alta quota, impone un ribaltamento dei paradigmi architettureali convenzionali. L'assunzione che la rete Internet sia costantemente disponibile risulterebbe fatale per Trento Smart Mountain. Di conseguenza, l'architettura è stata forgiata attorno al pattern **Offline-First**, promuovendo l'autonomia computazionale dei nodi periferici.

Attraverso l'applicazione rigorosa del principio di *Store-and-Forward*, i dati critici – quali le transazioni di gamification (gestite tramite un modello di **Event Sourcing**), i log di telemetria dei macchinari e i messaggi di chat – vengono immagazzinati localmente in database embedded cifrati (es. SQLite/-Room per il layer Mobile). Un componente specializzato, il **SyncManager**, monitora lo stato del network layer e, non appena un canale di comunicazione (4G/5G o Wi-Fi del rifugio) torna disponibile, provvede all'inoltro in **batch** dei dati verso il cloud. Tale approccio garantisce il teorema CAP posizionandosi strategicamente sul versante dell'Availability e della Partition Tolerance (AP), delegando la Consistency a un modello di *Eventual Consistency* gestito deterministicamente dal backend.

L'infrastruttura di sicurezza e accesso integra un **Gestore Autenticazione** basato sullo standard OAuth2/OpenID Connect. Per ovviare alla mancanza di rete, il sistema implementa un meccanismo di *Offline Token Caching*: una volta effettuata l'autenticazione iniziale a valle, i token JWT (JSON Web Token) e i profili crittografici vengono memorizzati nel *Secure Storage* del dispositivo, consentendo l'accesso biometrico e la validazione delle operazioni anche in totale isolamento. Parallelamente, il **Gestore Checklist** permette il download preventivo di mappe tecniche (OSM) e previsioni meteo, garantendo che le informazioni vitali per la sicurezza siano fruibili indipendentemente dalla copertura cellulare.

L'inizializzazione della "Sessione Escursione" (il gruppo) risolve il problema della formazione decentralizzata in assenza di segnale. Il Capogruppo genera un **QR Code Dinamico** contenente l'ID di sessione e la propria chiave pubblica ECC. Gli escursionisti, tramite scansione ottica, eseguono un *Handshake Crittografico Offline* che permette di instaurare la fiducia reciproca necessaria per la validazione dei messaggi nella rete mesh, senza scambiare un singolo byte con il cloud durante la fase di avvio sentiero.

Il sistema si articola su due rami operativi distinti: un ramo **B2B (Business-to-Business)** rivolto ai rifugi per la gestione logistica dei rifiuti e il monitoraggio dell'affollamento tramite **Edge Computing**, e un ramo **B2C (Business-to-Consumer)** incentrato sulla gamification, la sensibilizzazione ambientale, la verifica telemetrica dell'affluenza e la **sicurezza**. I rifugi alpini verranno equipaggiati con *IoT Edge Gateway* che fungono da orchestratori locali, aggregando i dati dai macchinari di trattamento (es. compattatori e disidratatori) e dai sensori. Invece di saturare la banda con flussi grezzi, l'Edge Gateway invia unicamente *telemetry summaries* e alert di stato. Per quanto riguarda la gestione dei gruppi, si è scelto di eliminare la dipendenza da API di verifica esterne (es. registri CAI) non accessibili: il ruolo di **Capogruppo** viene assegnato dinamicamente all'utente che inizializza la sessione, rendendolo il nodo relay primario.

Un terzo pilastro risiede nell'infrastruttura di emergenza ibrida. In caso di *shadowing elettromagnetico* (not-spot), il segnale SOS (contenente Nome, Coordinate GPS e tipologia di emergenza) viene firmato tramite ECC e propagato via **BLE Mesh** fino al Capogruppo. Quest'ultimo, agendo da primo soccorritore, valuta la gravità tramite la propria Dashboard operativa: può decidere per un *Intervento Diretto* (navigando verso l'infortunato) o per il *Relay Esterno*, inoltrando i dati alle autorità tramite il proprio hardware satellitare/radio professionale. Per garantire la resilienza, il sistema implementa un protocollo di **Failover a 10 minuti**: se il Capogruppo risulta irraggiungibile per la rete mesh, i privilegi di relay vengono delegati automaticamente applicando un algoritmo di elezione a cascata che privilegia la minore distanza geospaziale, il maggior livello di batteria residua, il ranking di Social Credits (S_c) più elevato e, in caso di ulteriore parità, l'ID utente minore.

1.2 Descrizione Analitica e Diagramma dei Componenti (UML)

Mentre l'infrastruttura di base garantisce la resilienza del dato, la logica di business di Trento Smart Mountain è distribuita in moduli funzionali che orchestrano l'esperienza dell'utente e del gestore. Il seguente diagramma illustra la topologia del sistema, evidenziando l'integrazione dei moduli di gestione identità, dati ambientali e sicurezza.

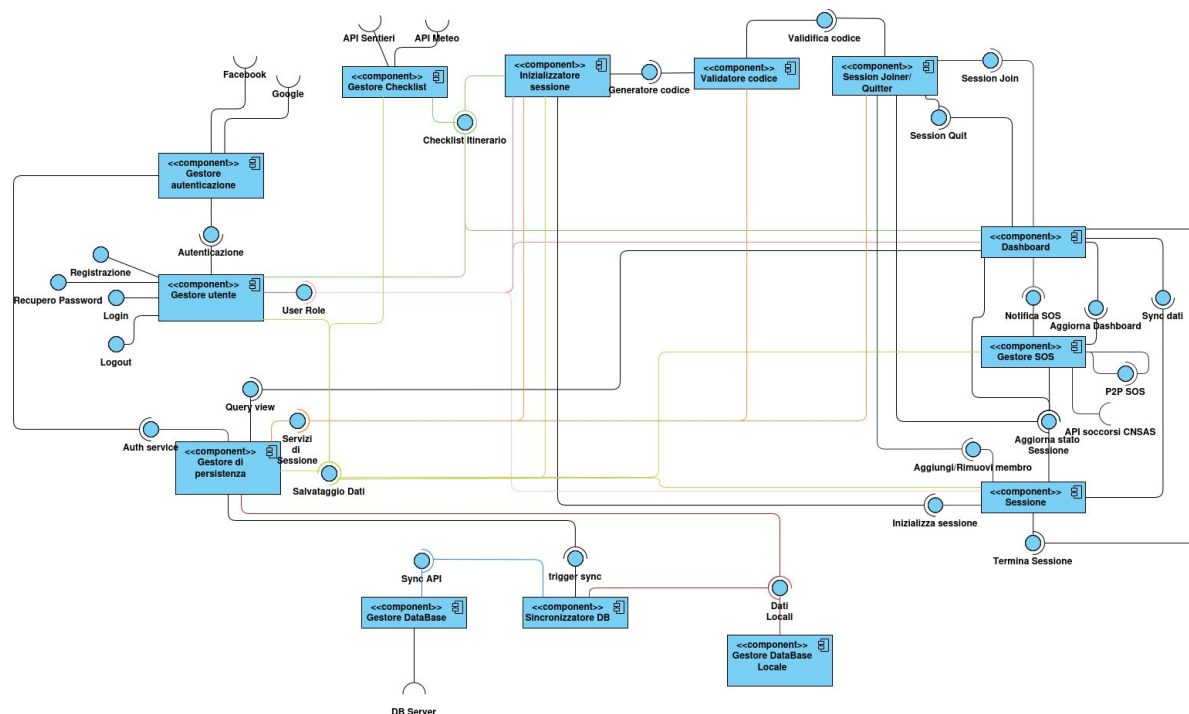


Figura 1: Diagramma UML dei Componenti

1.2.1 Gestore Autenticazione

Il modulo gestisce l'identità digitale integrando il protocollo OAuth2/OIDC. Per operare in contesti montani, il componente implementa il **Local Token Caching**: una volta validato il profilo online (es. via Google o Facebook), le credenziali e il ruolo (Escursionista/Capogruppo) vengono salvati in modo persistente nel *Secure Storage* del dispositivo, consentendo l'accesso e la firma crittografica delle operazioni anche in assenza di rete.

Gestore Autenticazione

Tipologia	Nome	Descrizione
Richiesta	Facebook / Google	Interfaccia Richiesta (Required Interface) i servizi esterni per l'autenticazione.
Fornita	Autenticazione	Interfaccia di risposta verso il Gestore Utente.
Richiesta	Auth Service	Richiede al Mediatore la validazione dei token.

1.2.2 Gestore Utente

Gestisce il profilo e i privilegi operativi (Guida/Escursionista). Definisce l'accesso alle funzionalità della Dashboard in base ai dati recuperati dal sistema di persistenza.

Gestore Utente

Tipologia	Nome	Descrizione
Fornita	Registrazione/Login	Interfacce per l'accesso e la creazione dell'account.
Fornita	Logout/Recupero Pw	Gestione della sicurezza del profilo.
Fornita	User Role	Espone il ruolo dell'utente alla Dashboard e all'Inizializzatore.
Richiesta	Autenticazione	Riceve i dati verificati dal Gestore Autenticazione.
Richiesta	Salvataggio Dati	Richiede al Mediatore il salvataggio dei dati del profilo utente.

1.2.3 Gestore Checklist

Organizza l'itinerario e il materiale. Recupera dati esterni e comunica con l'Inizializzatore per definire i dettagli tecnici della spedizione attraverso l'interfaccia Checklist Itinerario. All'attivazione della sessione (fase online), il gestore interroga le API cartografiche e meteo per scaricare l'**Hike Packet**: un bundle contenente la traccia GeoJSON e le **Map Tiles OSM** con un raggio di **padding di 1 km**, garantendo la consultazione cartografica e dei punti di interesse (POI) in zona di not-spot.

Gestore Checklist

Tipologia	Nome	Descrizione
Fornita	Checklist Itinerario	Dettagli del percorso forniti all'Inizializzatore Sessione.
Richiesta	API Sentieri/Meteo	Interfaccia Richiesta (Required Interface) per dati cartografici e previsioni aggiornate.
Richiesta	Salvataggio Dati	Richiede al Mediatore il salvataggio della checklist.

1.2.4 Inizializzatore Sessione

Componente che trasforma un utente in Capogruppo aprendo una nuova istanza di escursione. Comunica con il Mediatore per registrare l'inizio dell'attività.

Inizializzatore Sessione

Tipologia	Nome	Descrizione
Fornita	Generatore Codice	Crea il codice alfanumerico necessario per l'unione del gruppo.
Richiesta	Inizializza sessione	Configura la nuova sessione nel componente di controllo Sessione.
Richiesta	Servizi di Sessione	Richiede al Mediatore di registrare la nuova sessione.
Richiesta	Checklist Itinerario	Recupera i dettagli tecnici dal Gestore Checklist.

1.2.5 Validatore Codice

Verifica l'integrità dei codici inseriti dagli escursionisti per unirsi a una sessione esistente.

Validatore Codice

Tipologia	Nome	Descrizione
Fornita	Validazione Codice	Fornisce l'esito della validazione al Session Joiner.
Richiesta	Generatore Codice	Ottiene il codice creato dall'Inizializzatore per il confronto.
Richiesta	Servizi di sessione	Interagisce con il Mediatore per confermare l'esistenza della sessione.

1.2.6 Session Joiner/Quitter

Gestisce l'unione e l'abbandono fisici dell'utente dalla sessione. Dopo la validazione del codice, richiede al Gestore di Persistenza di aggiornare l'appartenenza dell'utente nel database.

Session Joiner/Quitter

Tipologia	Nome	Descrizione
Fornita	Session Join	Interfaccia esposta alla Dashboard per l'inserimento del codice e l'unione.
Fornita	Session Quit	Interfaccia esposta alla Dashboard per l'abbandono della sessione.
Richiesta	Validazione Codice	Richiede la verifica del codice inserito.
Richiesta	Servizi di Sessione	Richiede al Mediatore di aggiornare il database locale.
Richiesta	Aggiungi/Rimuovi membro	Notifica al componente Sessione la variazione dei partecipanti.

1.2.7 Sessione

Gestisce lo stato "live" dell'escursione e la coesione del gruppo. A differenza dei sistemi web-centrici, la Sessione è un'entità logica decentralizzata. Il modulo gestisce l'unione dei partecipanti tramite **Link/Codice (Online)** o **QR Code Handshake (Offline)**. In quota, il componente mantiene lo stato di coesione del gruppo (prossimità dei membri) sfruttando i beacon della rete mesh, senza richiedere il polling a un database remoto.

Sessione

Tipologia	Nome	Descrizione
Fornita	Inizializza sessione	Riceve il comando di creazione dall'Inizializzatore.
Fornita	Aggiungi/Rimuovi membro	Aggiorna la lista dei partecipanti attivi.
Fornita	Termina Sessione	Chiude formalmente l'escursione registrando i dati finali.
Fornita	Aggiorna stato Sessione	Notifica in tempo reale alla Dashboard i cambi di stato del gruppo.
Richiesta	Servizi di Sessione	Interfaccia Richiesta (Required Interface) il Mediatore per il salvataggio dei log di tracking.

1.2.8 Gestore SOS

Componente critico per la sicurezza. Inoltre l'allarme contenente Nome Utente, Coordinate GPS e tipo di emergenza via rete (se disponibile) o via protocollo mesh proprietario P2P verso il Capogruppo. Gestisce inoltre il protocollo di failover automatico (10 min) per la delega dei privilegi di relay in caso di leader offline. Il messaggio si propaga a macchia d'olio tra i nodi della rete locale fino a raggiungere il Capogruppo, l'unico abilitato al relay satellitare verso i soccorsi (CNSAS).

Gestore SOS

Tipologia	Nome	Descrizione
Fornita	Notifica SOS	Riceve l'attivazione dell'allarme dalla Dashboard locale.
Fornita	Aggiorna Dashboard	Invia alert visivi con dati infortunato alla UI.
Fornita	P2P SOS	Interfaccia mesh per la propagazione dell'allarme tra i dispositivi.
Richiesta	API soccorsi CNSAS	Canale di inoltro (via relay hardware) verso le autorità.
Richiesta	Salvataggio Dati	Registrazione dell'evento nel database locale tramite il Mediatore.

1.2.9 Gestore di Persistenza

Il mediatore centrale ("imbuto") per l'accesso ai dati. Implementa la logica *Local-First*, coordinando il database locale e attivando la sincronizzazione cloud tramite il modulo dedicato. Implementa la logica di prioritizzazione del dato: ogni operazione di scrittura viene prima validata e salvata localmente nel database embedded (SQLite/Room), garantendo l'immediata disponibilità dell'app; simultaneamente, il modulo decide se attivare il trigger di sincronizzazione verso il cloud in base alla telemetria del network layer. Al fine di mitigare il rischio di trasformare il mediatore (*gestore di persistenza*) in una "God Class" ad alto accoppiamento, l'implementazione segue i principi della Clean Architecture: il componente orchestra le operazioni delegando la logica decisionale sulla priorità dei dati e sul timing di invio a un componente specializzato **SyncEngine** (che implementa una *PolicyStrategy*), adottando inoltre un Repository Pattern che disaccoppia l'accesso ai dati tramite adapter specifici per ogni dominio funzionale.

Gestore di Persistenza

Tipologia	Nome	Descrizione
Fornita	Auth service	Interfaccia per la gestione delle credenziali.
Fornita	Servizi di Sessione	Gestione della persistenza per dati di gruppo e tracking.
Fornita	Salvataggio Dati	Interfaccia CRUD generica per i moduli funzionali.
Fornita	Query view	Interfaccia di sola lettura per la Dashboard (cache locale).
Richiesta	Dati Locali	Richiede al Gestore DataBase Locale la scrittura fisica.
Richiesta	trigger sync	Segnale di avvio per il modulo di sincronizzazione.

1.2.10 Sincronizzatore Db

Responsabile Del Disallineamento E Della Riconciliazione Tra I Dati Locali E Il Cloud.

Sincronizzatore Db

Tipologia	Nome	Descrizione
Fornita	Trigger Sync	Riceve Il Comando Di Allineamento Dal Mediatore.
Fornita	Sync Dati	Invia Aggiornamenti Di Sincronizzazione Alla Dashboard.
Richiesta	Sync Api	Connessione Verso Il Gestore Database Cloud Centrale.
Richiesta	Dati Locali	Lettura E Scrittura Sul Database Fisico Locale.

1.2.11 Gestore DataBase

Infrastruttura cloud centrale per la persistenza a lungo termine e la sincronizzazione globale.

Gestore DataBase

Tipologia	Nome	Descrizione
Fornita	Sync API	Interfaccia REST per la sincronizzazione dei client.
Richiesta	DB Server	Connessione al cluster MongoDB remoto.

1.2.12 Gestore DataBase Locale

Storage fisico (SQLite/Room) sul dispositivo mobile.

Gestore DataBase Locale

Tipologia	Nome	Descrizione
Fornita	Dati Locali	Interfaccia di I/O utilizzata dal Mediatore e dal Sincronizzatore.

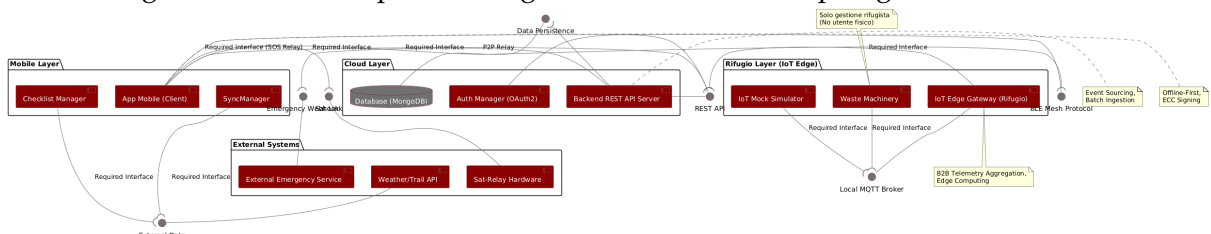
1.2.13 Dashboard

Rappresenta il punto di interazione unificato e adattivo. La UI si adatta dinamicamente allo stato della connessione e al ruolo dell'utente, esponendo gli strumenti di navigazione, i quiz di gamification e la segnalazione dei sentieri (Path Reporting) in modo trasparente rispetto alla complessità del layer di sincronizzazione sottostante, consumando i dati dalla cache locale per garantire fluidità totale.

Dashboard

Tipologia	Nome	Descrizione
Richiesta	Query view	Recupera lo stato della sessione e del profilo dal Mediatore.
Richiesta	Session Join/Quit	Trigger per l'unione o l'abbandono del gruppo.
Richiesta	Notifica SOS	Comando di attivazione dell'allarme emergenza.
Richiesta	Aggiorna Dashboard	Riceve notifiche push locali dal Gestore SOS.
Richiesta	Aggiorna stato Sessione	Riceve dati live sull'escursione dal componente Sessione.
Richiesta	Sync dati	Riceve lo stato della sincronizzazione dal Sincronizzatore.

Figura 2: UML Component Diagram - Interazione e Topologia Architetturale



1.3 Dizionario delle Interfacce e Protocolli

La matrice seguente formalizza i contratti (*Interfaces*) tra i componenti del sistema. Seguendo la notazione UML, vengono distinti i fornitori (*Realization*) dai consumatori (*Usage*), garantendo un disaccoppiamento ottimale tra i moduli.

Interface	Supplier	Applicant	Protocol	Contract & Function
IRestApi	Backend Server	App Mobile, IoT Gateway	HTTPS	Definisce gli endpoint per la sincronizzazione batch e l'ingestione della telemetria aggregata.
IAuth	Auth Manager	App Mobile	OAuth2 / JWT	Contratto per lo scambio credenziali e il rilascio dei token necessari per l'operatività offline.
ILocalMqtt	IoT Gateway	Macchinari, Sensori	MQTT	Broker locale per la raccolta dati grezzi dai sensori industriali e dai sensori ottici nel rifugio.
IMeshSOS	App Mobile	App Mobile (Relay)	BLE ADV	Protocollo di propagazione <i>connectionless</i> per l'SOS a macchia d'olio con prevenzione dei loop.
ISatLink	External Sat Hardware	Capogruppo (App)	BLE/ Serial	Interfaccia di relay per l'inoltro di segnali emergenziali tramite hardware professionale esterno.
IExternalData	Weather/Trail API	Checklist Manager	REST	Contratto per il download preventivo dell'Hike Packet (itinerari e tiles cartografiche).

2 Specifica delle API RESTful

La progettazione delle interfacce *Application Programming Interface* (API) di Trento Smart Mountain aderisce all'architettura software **REST** (Representational State Transfer). Il principio di *Statelessness* è categoricamente rispettato: ogni singola request JSON contiene tutte le informazioni (es. Bearer Token JWT) necessarie affinché il backend possa validare l'operazione. La granularità degli HTTP Status Code viene sfruttata come canale primario per il *fault tolerance*, permettendo al *SyncManager* del layer mobile di gestire le sincronizzazioni in batch via **Store-and-Forward**.

2.1 Endpoint: Autenticazione e Rilascio Token

Metodo e URI	POST /api/v1/auth/login
Descrizione	Scambio delle credenziali (o token di terze parti OAuth2) per un JWT firmato dal server, necessario per l'operatività offline successiva.

Listing 1: Response Payload - Successo (200 OK)

```
1  {
2    "accessToken": "eyJhbGciOiJIUzI1Ni...",
3    "refreshToken": "def56789...",
4    "expiresIn": 3600,
5    "userProfile": {
6      "_id": "usr_78a9b",
7      "role": "ESCURSIONISTA",
8      "eccPublicKey": "0450863ad64a87..."
9    }
10 }
```

2.2 Endpoint: Ricezione SOS (ECC Verified)

Metodo e URI	POST /api/v1/emergencies
Descrizione	Ingestione di un segnale SOS. Il backend valida la firma crittografica ECC generata dal dispositivo sorgente per prevenire falsi allarmi.

Listing 2: Request Payload - SOS Mesh Relay

```
1  {
2    "emergencyType": "MEDICAL_ASSISTANCE",
3    "issuerId": "usr_78a9b_beta",
4    "payload": {
5      "userName": "Giacomo Radin",
6      "lat": 46.123456,
7      "lng": 11.234567,
8      "timestamp": "2026-04-18T14:45:32Z"
9    },
10   "eccSignature": "3045022100ed...",
11   "meshMetadata": { "hopCount": 3, "relayNodeId": "usr_caiguide_01" }
12 }
```

2.3 Endpoint: Ingestione Telemetria (Batch)

Metodo e URI	POST /api/v1/rifugi/{id}/telemetry/batch
Descrizione	Ingestione massiva di log accumulati durante periodi di assenza di rete dall'Edge Gateway.

Listing 3: Request Payload - Batch IoT

```
1  {
2    "gatewayId": "gw_tuckett_01",
3    "events": [
4      { "machineryId": "COMP_01", "metrics": { "vol": 74.5 }, "ts":
5        "..."},
6      { "machineryId": "COMP_01", "metrics": { "vol": 78.2 }, "ts":
7        "..."}
8    ]
9  }
```

2.4 Endpoint: Sync Gamification (Event Sourcing)

Metodo e URI	POST /api/v1/users/{id}/gamification/sync
Descrizione	Il SyncManager invia la lista di eventi guadagnati offline. Il server li applica deterministicamente per risolvere conflitti.

Listing 4: Request Payload - Event Log Sync

```
1  {
2    "events": [
3      { "type": "QUIZ_COMPLETED", "sc": 50, "eventId": "ev_001", "ts":
4        "..."},
5      { "type": "NFC_SCAN", "sc": 100, "eventId": "ev_002", "ts": "..."}
6    ]
7  }
```

2.5 Gestione Errori e Transazioni Fallite

Per garantire la robustezza del sistema in scenari di connettività intermittente, le API adottano una politica rigorosa di gestione degli errori basata sulla semantica HTTP:

- **401 Unauthorized:** Token JWT scaduto o non valido. Il client deve tentare il refresh tramite le credenziali cachate offline o richiedere nuovo login al rientro in copertura.
- **403 Forbidden:** Azione non permessa per il ruolo attuale (es. Escursionista che tenta di resettare un macchinario IoT).
- **422 Unprocessable Entity:** Violazione dei vincoli di business (es. saldo crediti insufficiente o firma ECC non valida).
- **503 Service Unavailable:** Fallimento transitorio del relay satellitare. Il client deve applicare un algoritmo di *Exponential Backoff*.

3 Diagramma delle Classi e Modello Dati

La progettazione statica orientata agli oggetti rappresenta il nucleo semantico del software. L'architettura delle classi delineata di seguito modella un dominio complesso in cui entità fisiche, meccanismi di comunicazione differita ed elementi di gamification si fondono in un singolo ecosistema logico, aderendo ai requisiti di resilienza in contesti isolati.

3.1 Diagramma delle Classi (UML)

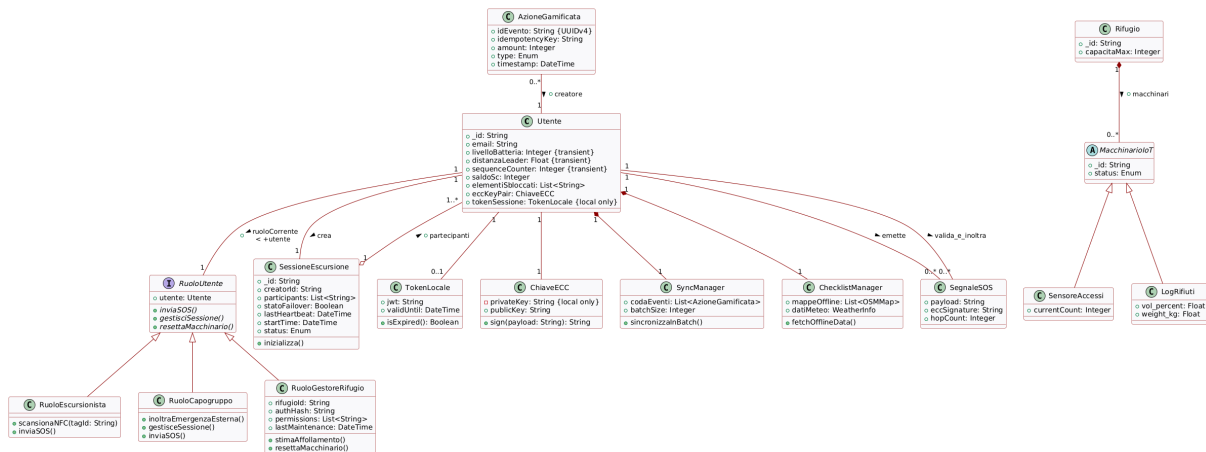


Figura 3: UML Class Diagram - Core Domain Model con integrazione Offline-First

3.2 Analisi Dettagliata dei Design Pattern e delle Scelte Architetture

3.2.1 Event Sourcing per la Sincronizzazione Batch

Per garantire la consistenza del saldo crediti (S_c) senza richiedere una connessione costante, il sistema adotta il pattern **Event Sourcing**. Invece di memorizzare lo stato corrente sul client, il **SyncManager** registra una sequenza immutabile di oggetti **AzioneGamificata**. Quando il device torna online, il backend non sovrascrive il valore finale, ma esegue il "replay" degli eventi ricevuti in batch, garantendo l'integrità atomica e risolvendo i conflitti tramite l'ordinamento temporale degli eventi.

3.2.2 Sicurezza Distribuita via ECC (Elliptic Curve Cryptography)

L'assenza di un server di autenticazione centrale durante l'escursione espone il sistema a rischi di "spoofing". La classe **ChiaveECC** implementa la crittografia asimmetrica su curve ellittiche utilizzando lo standard **Ed25519**, scelto per l'elevata velocità di firma e verifica su dispositivi mobile e IoT a basso consumo. All'emissione di un **SegnaleSOS**, il client genera una firma digitale del payload. Il Capogruppo, possedendo la chiave pubblica degli utenti della propria sessione (scambiata tramite un **Out-of-Band Key Exchange** durante l'inizializzazione online o via QR Code), può validare l'allarme offline con certezza matematica dell'identità del mittente, mitigando strutturalmente attacchi di tipo *Man-in-the-Middle* (MitM) nella rete mesh e garantendo l'integrità del dato prima di attivare i costosi canali di soccorso satellitari. A livello implementativo, l'attributo `privateKey` è escluso dai DTO di serializzazione, garantendo che non venga mai trasmesso o persistito sul MongoDB cloud, come confermato dallo schema della Sezione 8.

3.2.3 Pattern Proxy e Caching per Checklist e Auth

Il **ChecklistManager** e il **TokenLocale** agiscono come **Proxy** verso le risorse remote. In presenza di rete, caricano i dati (mappe, meteo, JWT) dai server REST; in assenza di rete, servono le richieste dell'utente attingendo alla cache locale crittografata, mascherando l'interruzione del servizio al layer di presentazione.

3.2.4 Mapping Polimorfico (Strategy vs Document)

Il sistema gestisce la polimorficità dei ruoli utente tramite un mapping bidirezionale tra il database orientato ai documenti e il modello a oggetti. All'interno della collezione `users`, il campo stringa `"role"` funge da *Discriminator* (es. "ESCURSIONISTA", "CAPOGRUPPO"). Il layer di persistenza (*Persistence Mediator*), durante la fase di *hydration* del documento JSON, utilizza una *factory* per istanziare la classe concreta corrispondente del pattern **Strategy** (`RuoloEscursionista`, `RuoloCapogruppo`), iniettando dinamicamente i comportamenti e i privilegi operativi richiesti senza violare il principio di incapsulamento.

3.2.5 Ruolo Dinamico del Capogruppo e Bootstrap Offline

In conformità con l'assenza di albi professionali digitalizzati, la relazione tra Capogruppo e `SessioneEscursione` è basata sul principio di *Ownership*. Chi istanzia l'oggetto `SessioneEscursione` assume automaticamente i metodi privilegiati di gestione e relay. Il processo di unione al gruppo (*Session Join*) avviene tramite un **Handshake Crittografico via Code**: il Capogruppo espone la propria chiave pubblica ECC nel codice, che i membri scansionano per configurare i nodi della rete mesh locale, garantendo la sicurezza del sistema senza necessità di server di discovery centralizzati in quota.

4 Formalizzazione dei Vincoli OCL (Object Constraint Language)

In questa sezione vengono formalizzati in linguaggio OCL i vincoli architetturali legati al diagramma delle classi. La natura distribuita e Offline-First di Trento Smart Mountain richiede vincoli rigorosi per garantire l'integrità transazionale, la sicurezza della rete mesh e la coerenza del layer IoT periferico. Vengono definiti i seguenti constraint, suddivisi per aree logiche di competenza:

4.1 Invarianti di Dominio: Gamification e Utenti

- **1. Lower-bound della Valuta Sociale**

Descrizione: Il saldo dei Crediti Sociali (Sc) di un utente non può mai assumere un valore negativo, assicurando la coerenza contabile in fase di Event Sourcing.

```
context Utente
inv NessunDeficitSociale: self.saldoSc >= 0
```

- **2. Coerenza Positiva della Gamification**

Descrizione: Qualsiasi azione gamificata registrata offline deve generare un punteggio strettamente positivo per essere considerata valida dal motore logico.

```
context AzioneGamificata
inv AssenzaPunteggiNulli: self.amount > 0
```

- **3. Idempotenza Locale degli Eventi**

Descrizione: Garantisce che la coda di sincronizzazione locale non contenga duplicati prima dell'invio in batch, utilizzando le chiavi di idempotenza generate dal client.

```
context SyncManager
inv IdempotenzaLocale:
    self.codaEventi->isUnique(idempotencyKey)
```

4.2 Invarianti di Sistema: Rete Mesh e IoT (Edge Computing)

- **4. Integrità Crittografica e TTL del Segnale SOS**

Descrizione: Un segnale SOS instradato nella rete BLE Mesh è valido se e solo se possiede una firma crittografica ECC definita e un Time-To-Live (hopCount) entro i limiti di flooding consentiti.

```
context SegnaleSOS
inv ValiditaECC:
    not self.eccSignature.oclIsUndefined() and
    self.eccSignature <> '' and
    self.hopCount <= 10
```

- **5. Validazione Range Sensore Ottico (Hardware IoT)**

Descrizione: I dati telemetrici inviati dai sensori di volume devono rientrare nel range percentuale valido (0-100%).

```
context LogRifiuti
inv CoerenzaRangePercentuale:
    self.vol_percent >= 0.0 and self.vol_percent <= 100.0
```

- **6. Limiti Matematici dell'Affollamento**

Descrizione: Il contatore logico dei sensori ottici di accesso al rifugio non deve mai scendere al di sotto dello zero.

```
context Rifugio
inv AffollamentoNonNegativo:
    self.macchinari->select(oclIsTypeOf(SensoreAccessi))
    ->forall(s | s.currentCount >= 0)
```

- **7. Unicità degli Identificatori Hardware (IoT Gateway)**

Descrizione: Tutti i macchinari e i sensori afferenti al nodo edge di uno specifico rifugio devono possedere un identificatore MAC/ID univoco all'interno di quella topologia.

```
context Rifugio
inv IdentificatoriHardwareUnici:
    self.macchinari->isUnique(_id)
```

- **8. Integrità Temporale della Sincronizzazione Batch**

Descrizione: Il buffer di messaggi in Store-and-Forward gestito dal componente di sincronizzazione non deve contenere eventi con timestamp situati nel futuro rispetto al clock di sistema.

```
context SyncManager
inv NoFutureEvents:
    self.codaEventi->forall(e | e.timestamp <= DateTime.now())
```

- **9. Elezione Leader Univoca (Failover)**

Descrizione: Durante uno stato di failover, il sistema deve garantire l'esistenza di un unico leader delegato per prevenire conflitti decisionali.

```
context SessioneEscursione
inv ElezioneLeaderUnivoca:
    self.statoFailover = true implies
    self.partecipanti->one(p | p.ruoloCorrente.oclIsTypeOf(
        RuoloCapogruppo))
```

- **10. Riconciliazione Leadership (Anti-Split-Brain)**

Descrizione: In caso di ricongiungimento di due partizioni mesh con leader distinti, il nodo con priorità inferiore abdica automaticamente applicando la cascata:
DistanzaLeader (min) → Batteria (max) → Sc (max) → ID (min).

```
context SessioneEscursione
inv ReconciliationPhase:
    self.partecipanti->select(p |
        p.ruoloCorrente.oclIsTypeOf(RuoloCapogruppo))
        ->size() > 1 implies
    self.partecipanti->select(p |
        p.ruoloCorrente.oclIsTypeOf(RuoloCapogruppo))
        ->forall(p1, p2 | p1 <> p2 implies
            p1.distanzaLeader < p2.distanzaLeader or (
                p1.distanzaLeader = p2.distanzaLeader and (
                    p1.livelloBatteria > p2.livelloBatteria or (
                        p1.livelloBatteria = p2.livelloBatteria and (
                            p1.saldoSc > p2.saldoSc or (
                                p1.saldoSc = p2.saldoSc and p1._id < p2._id))))))
```

*Nota tecnica: Vista l'assenza di uno stato globale in una rete mesh partizionata, l'invariante si applica alla **materialized view** locale del nodo durante la fase di riconciliazione: se un dispositivo rileva la presenza di due beacon di tipo leader, forza l'abdicazione del nodo con priorità inferiore nella propria tabella di routing.*

4.3 Precondizioni e Postcondizioni (Metodi e Operazioni)

- **11. Sicurezza Tipologica sul Controllo Hardware (RBAC)**

Descrizione: Il riavvio o il reset dei macchinari IoT è un'operazione transazionale riservata esclusivamente alle istanze con ruolo `GestoreRifugio`.

```
context RuoloGestoreRifugio::resettaMacchinario()  
pre AutorizzazioneGestore:  
    self.ruoloCorrente.oclIsTypeOf(RuoloGestoreRifugio)
```

- **12. Vincolo Dinamico e Failover per Relay Emergenze**

Descrizione: Failover timeout di 10 minuti. L'instradamento di un SOS verso i canali satellitari esterni (CNSAS) può essere innescato dal Capogruppo o, in caso di attivazione del FailoverMode, dal leader eletto.

```
context SessioneEscursione  
inv FailoverTimeout:  
    (DateTime.now() - self.lastHeartbeat).minutes >= 10 implies  
        self.statoFailover = true
```

- **13. Condizione di Boot per l'Inoltro Emergenze**

Descrizione: L'operazione di invio di un segnale SOS da parte dell'utente è condizionata alla validità del token crittografico di sessione locale.

```
context RuoloUtente::inviaSOS()  
pre TokenAttivo:  
    not self.utente.tokenSessione.oclIsUndefined() and  
    not self.utente.tokenSessione.isExpired()
```

- **14. Variazione di Stato Post-Inizializzazione**

Descrizione: L'invocazione riuscita del costruttore o metodo di setup della sessione di escursione impone al sistema una transizione di stato verso la condizione attiva.

```
context SessioneEscursione::inizializza()  
post StatoAttivo:  
    self.status = StatoSessione::ATTIVA
```

5 Modellazione Dinamica - Sequence Diagrams

In questa sezione vengono analizzati i flussi temporali e lo scambio di messaggi tra i componenti per gli scenari critici di emergenza e sincronizzazione.

5.1 Scenario SOS: Relay Mesh con Validazione ECC

Il diagramma illustra come un allarme emesso in un "not-spot" raggiunga i soccorsi tramite la validazione del Capogruppo.

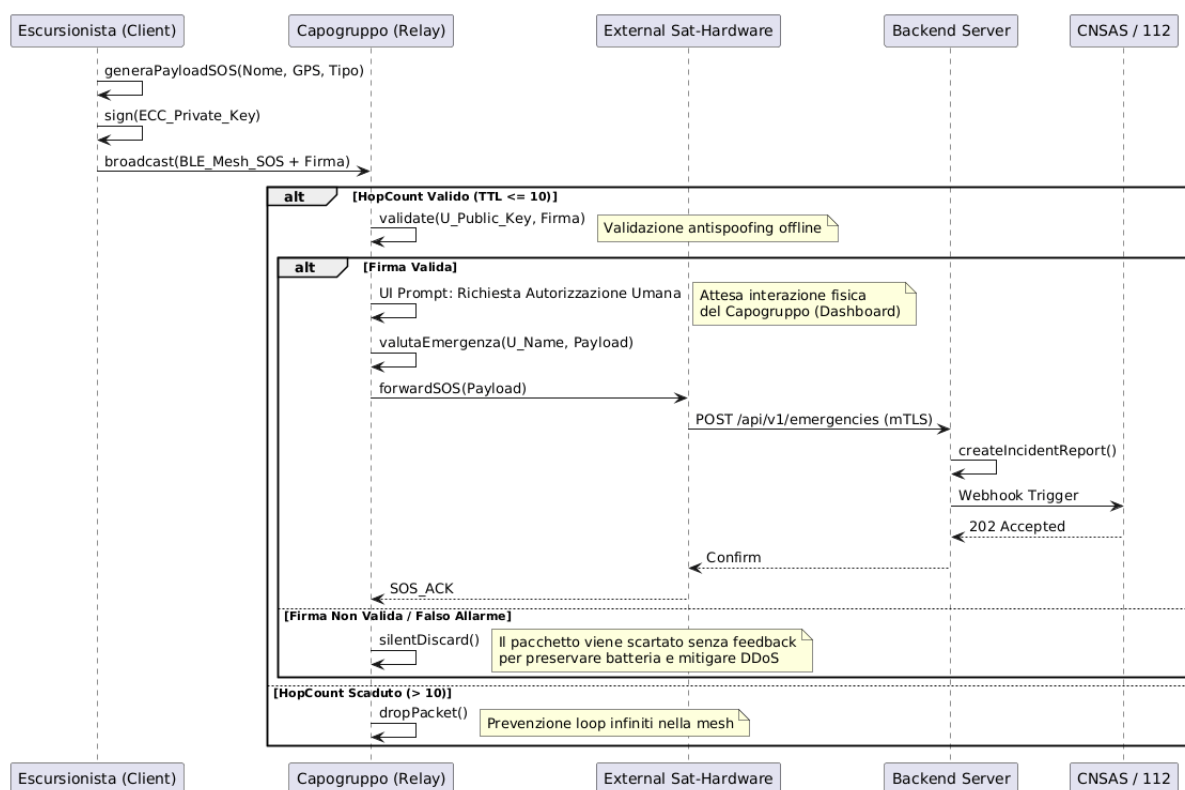


Figura 4: UML Sequence Diagram - Flusso Emergenza Ibrido

Il sistema risolve il paradosso del leader infortunato tramite un protocollo di delega logica: il failover elegge il membro più idoneo (per ranking o disponibilità energetica) a coordinare i soccorsi. Per l'inoltro satellitare, l'hardware esterno deve disporre di un tasto "Panic" fisico indipendente, oppure il nuovo leader dovrà raggiungere l'ultima posizione nota del predecessore per recuperare e operare il dispositivo di relay.

Nota tecnica: Il relay SOS non è automatico. Il sistema richiede l'approvazione esplicita del Capogruppo tramite Dashboard per prevenire falsi allarmi. In caso di errore di validazione, il sistema adotta una politica di `silentDiscard()`: il pacchetto viene scartato senza feedback per preservare la batteria del Capogruppo e mitigare tentativi di saturazione della rete. Il client mittente implementa un tetto massimo di tentativi (Max Retries) con backoff esponenziale per prevenire loop infiniti. Se l'infortunato è il Capogruppo stesso, il sistema prevede due vie: l'attivazione fisica del pulsante di panico sull'hardware satellitare da parte di un compagno, oppure l'attesa del Failover (10 min) per delegare il comando digitale al nuovo leader.

5.2 Scenario Sincronizzazione: Batch Sync & Event Sourcing

Descrizione del processo di persistenza transitoria e allineamento asincrono.

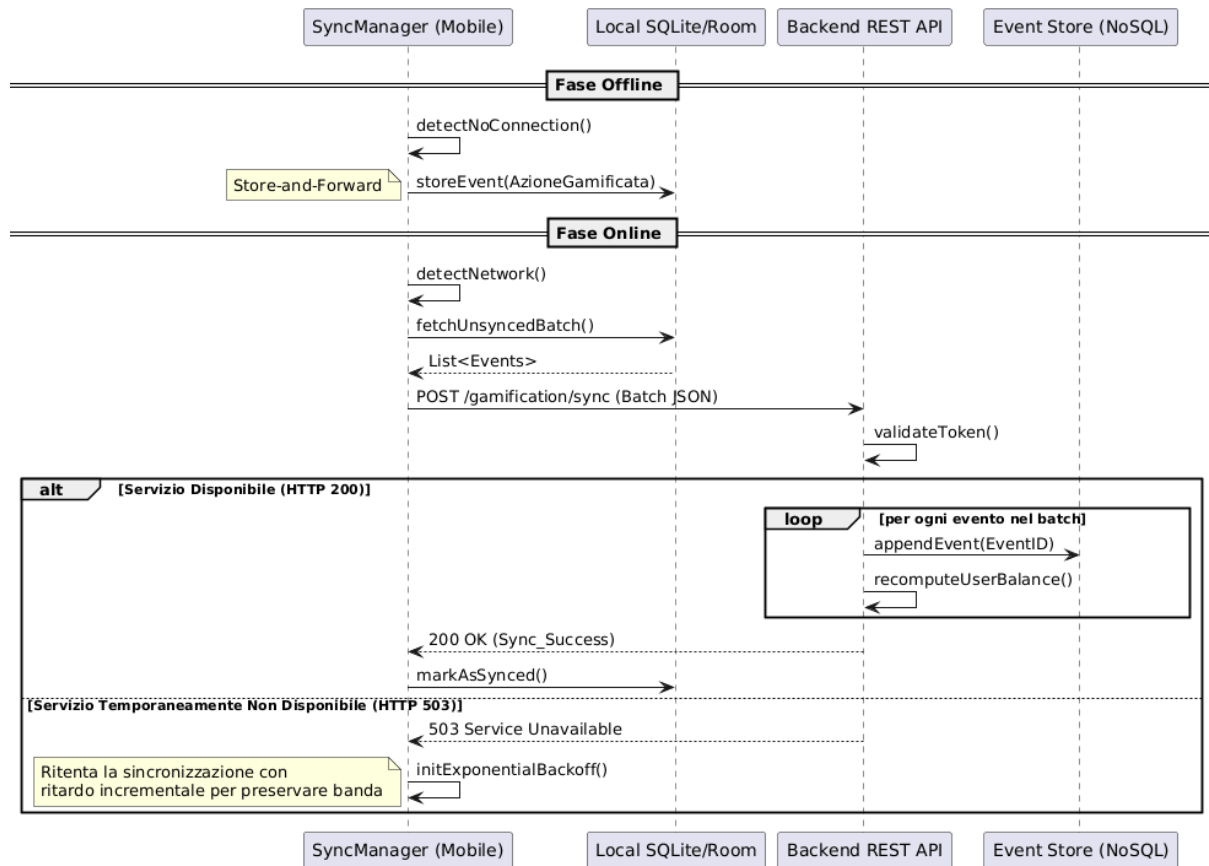


Figura 5: UML Sequence Diagram - Batch Sincronizzazione via Event Sourcing

6 Modellazione degli Stati - State Machine Diagram

La robustezza del sistema è garantita dalla gestione formale degli stati del client mobile, che deve adattare il proprio comportamento alla disponibilità energetica e telematica. Per preservare l'autonomia energetica dei dispositivi (vincolo critico in quota), il sistema adotta un protocollo di monitoraggio passivo. Il Capogruppo non emette heartbeat attivi, ma sfrutta il BLE Advertising a basso duty-cycle (simile allo standard iBeacon). Il protocollo di Failover viene innescato passivamente dai membri del gruppo solo dopo l'assenza prolungata (10 minuti) di pacchetti pubblicitari dal leader, minimizzando l'uso della radio.

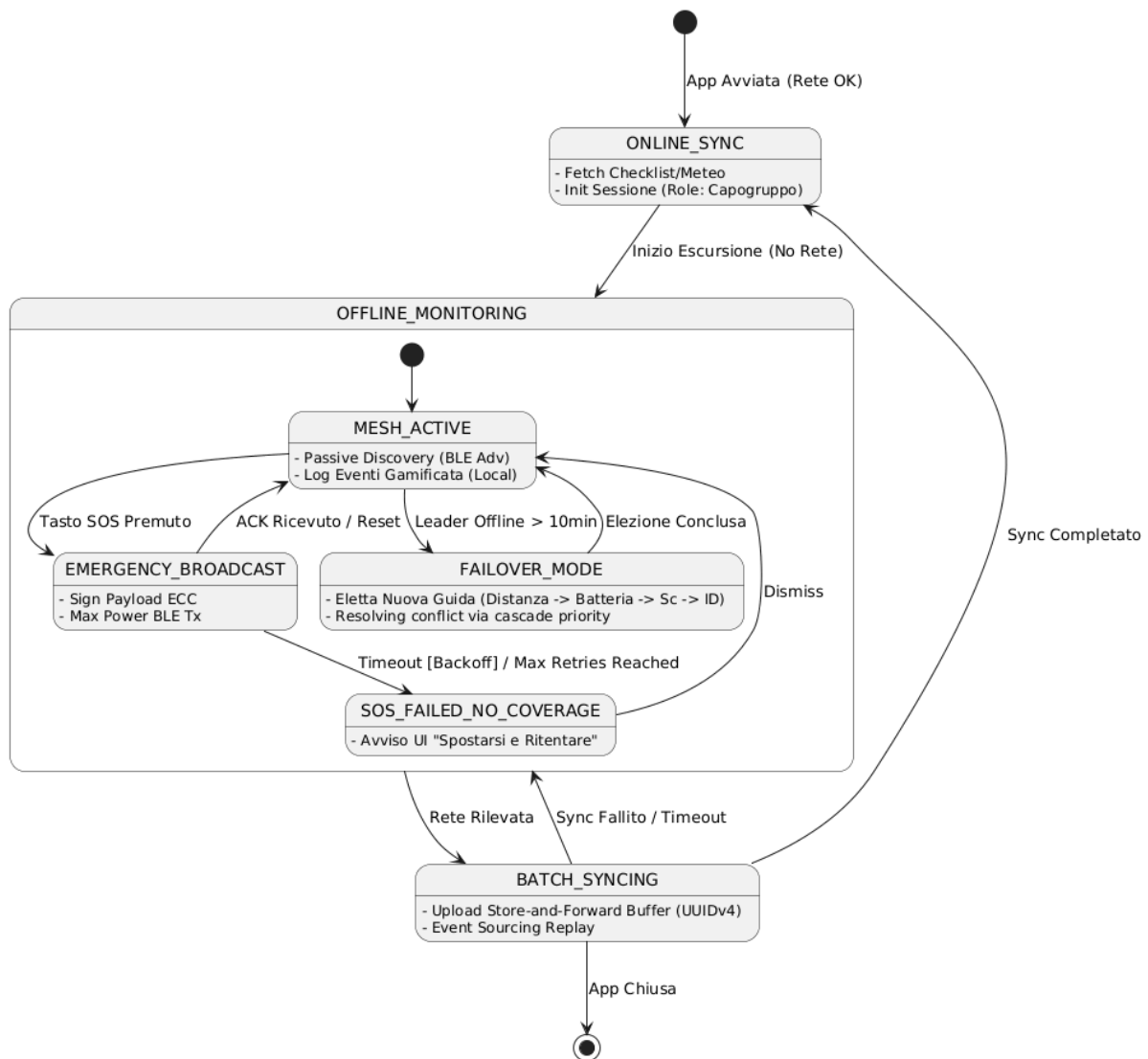


Figura 6: UML State Machine - Ciclo di Vita del Client Mobile in Quota

ELECTION: Risolve i conflitti tramite la cascata di priorità (Distance $\Rightarrow$ Battery $\Rightarrow$ Sc $\Rightarrow$ ID). Il protocollo di Failover si attiva quando l'assenza di beacon supera il threshold di 10 minuti per assicurare la resilienza contro il clock drift.

7 Architettura Fisica e Deployment Diagram

Il diagramma di distribuzione (UML Deployment Diagram) illustra l'allocazione fisica dei componenti software sui nodi hardware, evidenziando la natura distribuita e gerarchica dell'infrastruttura.

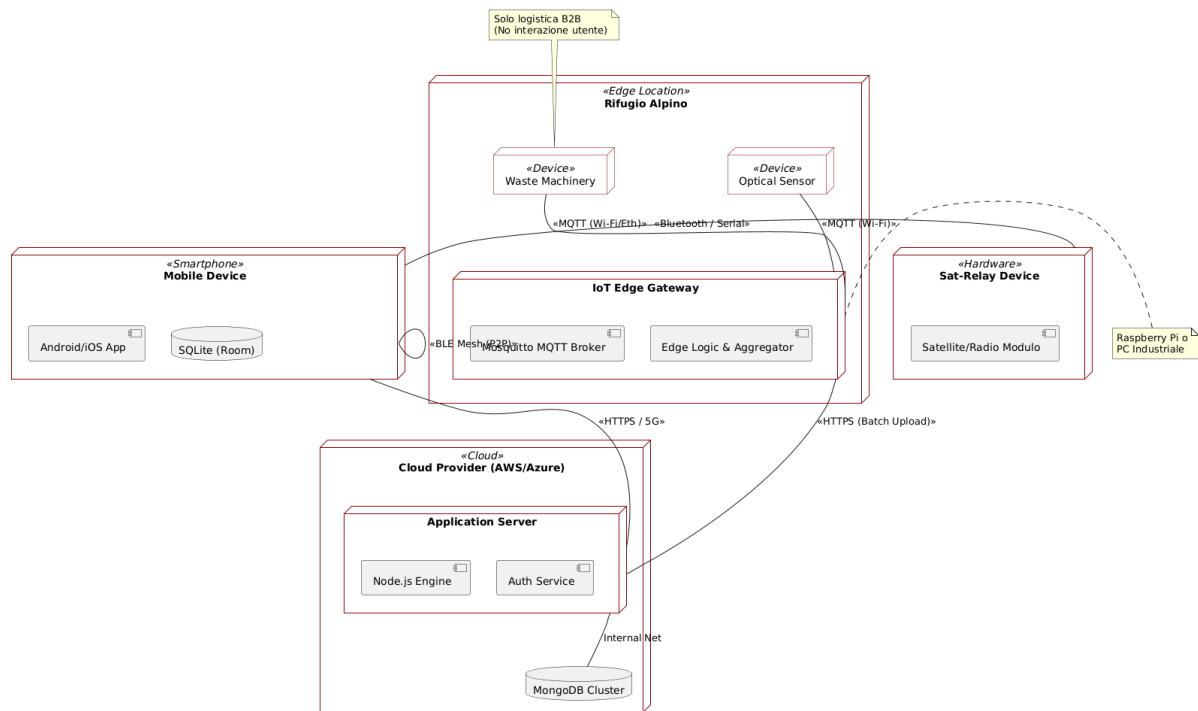


Figura 7: UML Deployment Diagram - Distribuzione Fisica dei Nodi

Nota sulla Privacy (Edge AI): Il Sensore Ottico effettua l'inferenza e il conteggio delle sagome direttamente on-board (Edge Computing). Nessun flusso video viene trasmesso sul broker MQTT, garantendo la totale compliance GDPR (Privacy-by-Design).

8 Modello Dati Fisico - MongoDB Document Schema

In conformità con la scelta di un database NoSQL per il backend, la persistenza viene modellata tramite documenti JSON denormalizzati, garantendo una separazione netta tra i privilegi operativi (RBAC) e l'integrità temporale degli eventi.

Listing 5: MongoDB Collection: users (Escursionisti e Guide)

```
1 {
2   "_id": "usr_78a9b",
3   "email": "giacomo.radin@example.com",
4   "role": "CAPOGRUPPO",
5   "eccKeyPair": {
6     "publicKey": "0450863ad64a87..."
7   },
8   "saldoSc": 2450,
9   "sequenceCounter": 142,
10  "elementiSbloccati": ["badge_marmolada"]
11 }
```

Nota: Al fine di mitigare la *Write Contention*, i dati volatili di sessione (Token JWT, Expiry e Session ID corrente) sono gestiti tramite un layer di caching distribuito (Redis) o risolti tramite logica stateless JWT, scorporandoli dal documento anagrafico dell'utente.

Listing 6: MongoDB Collection: managers (Gestori Rifugio)

```
1 {
2   "_id": "mgr_rif_01",
3   "rifugioId": "RIF_TUCKETT_001",
4   "authHash": "$2b$12$...",
5   "permissions": ["RESET_HARDWARE", "ACCESS_ANALYTICS"],
6   "lastMaintenance": "2025-12-10"
7 }
```

Listing 7: MongoDB Collection: user_event_store

```
1 {
2   "userId": "usr_78a9b",
3   "sequenceNumber": 143,
4   "idEvento": "UUIDv4",
5   "type": "SOCIAL_CREDIT_EARNED",
6   "amount": 100,
7   "timestamp": "2026-04-19T15:30:00Z",
8   "idempotencyKey": "v4_9921_01"
9 }
```

Nota: L'uso di UUID v4 garantisce l'univocità degli eventi su scala globale senza collisioni, mentre un sequenceNumber locale garantisce l'integrità dell'ordinamento per singolo utente. Il sistema adotta il pattern **CQRS (Command Query Responsibility Segregation)**: la collection `user_event_store` funge da Write-Model immutabile, mentre il campo `saldoSc` nella collection `users` rappresenta il Read-Model (Materialized View). Per garantire la coerenza tra i due modelli, il backend utilizza **Transazioni Multi-Documento** (MongoDB 4.0+) che assicurano l'atomicità (ACID) tra l'inserimento dell'evento e l'aggiornamento del saldo.

Listing 8: MongoDB Collection: hike_sessions

```
1 {
2   "_id": "hike_ses_202",
3   "creatorId": "usr_78a9b",
4   "status": "COMPLETED",
5   "statoFailover": false,
6   "lastHeartbeat": "2026-04-18T17:40:00Z",
7   "participants": ["usr_78a9b", "usr_marco_11"],
8   "startTime": "2026-04-18T08:00:00Z"
9 }
```

Nota: Per prevenire conflitti durante la sincronizzazione di unioni offline multiple, gli aggiornamenti all'array `participants` vengono eseguiti tramite operatori atomici di MongoDB (es. `$addToSet`), garantendo l'integrità del dato senza necessità di lock distribuiti.

Listing 9: MongoDB Collection: telemetry_logs (IoT Time-Series)

```
1 {
2   "_id": "log_662a1",
3   "gatewayId": "gw_tuckett_01",
4   "machineryId": "COMP_PET_01",
5   "rifugioId": "RIF_TUCKETT_001",
6   "metrics": {
7     "vol_percent": 62.1,
8     "weight_kg": 12.4,
9     "status": "OPERATIONAL"
10  },
11   "timestamp": "2026-04-21T10:00:00Z",
12   "metadata": { "batchId": "batch_v1_009" }
13 }
```

9 Specifiche dei Protocolli di Comunicazione

9.1 Infrastruttura MQTT (Publish/Subscribe)

La comunicazione locale nel rifugio segue una gerarchia di topic standardizzata per garantire l'estensibilità:

- `rifugio/{id}/status`: Stato vitale del gateway.
- `rifugio/{id}/macchinario/{mid}/telemetry`: Payload dati grezzi.
- `rifugio/{id}/macchinario/{mid}/cmd`: Comandi di reset (solo per Gestore).

Listing 10: Esempio Payload MQTT - Telemetria Compattatore

```
1 {
2   "mid": "COMP_PET_01",
3   "status": "OPERATIONAL",
4   "data": {
5     "vol_percent": 62.1,
6     "weight_kg": 12.4,
7     "temp_c": 24.5
8   },
9   "ts": 1713621600
10 }
```

9.2 Profilo BLE GATT per SOS

Il segnale di emergenza offline viene propagato tramite un servizio GATT custom definito come segue:

UUID Servizio	Caratteristica	Descrizione Payload
0xFD12	SOS_DATA (Read/Notify)	Coordinate GPS (Lat/Lng) e Timestamp UTC.
0xFD13	SOS_AUTH (Write)	Firma crittografica ECC (64 bytes).
0xFD14	SOS_STATUS (Read)	Stato del Relay (PENDING / FORWARDED).

Tabella 1: Specifiche del Profilo Bluetooth Low Energy per Emergenze

10 Infrastruttura Interna e Roadmap Sviluppo

10.1 Internal Event Bus (Backend Disaccoppiato)

Al fine di garantire la manutenibilità del Monolito Modulare in Node.js, la comunicazione tra i moduli (IoT, Gamification, Auth) avviene tramite un **Internal Event Bus** (basato su `EventEmitter`).

- Il modulo `Emergency` pubblica l'evento `SOS_RECEIVED`.
- Il modulo `Notification` ascolta l'evento e inoltra il push al Capogruppo.
- Questo disaccoppiamento permette una futura migrazione a Microservizi tramite l'inserimento di un broker esterno (es. Redis) senza modifiche alla logica di business.

10.2 Idempotenza e Integrità Transazionale

Per prevenire il *double-spending* dei Social Credits dovuto a ritrasmissioni del `SyncManager`, ogni transazione client-side genera obbligatoriamente un **Idempotency Key (UUID v4)**. Il backend persiste tali chiavi in un indice univoco su MongoDB; tentativi di invio di eventi già processati vengono scartati con HTTP 200 (idempotenza semantica). Per evitare un degrado delle performance del database nel lungo periodo, sulla collezione delle chiavi di idempotenza è configurato un **TTL Index (Time-To-Live)** su MongoDB che elimina automaticamente i record più vecchi di 30 giorni.

10.3 Roadmap Implementativa (Sprint 1)

La fase di coding inizierà con la realizzazione del **SyncManager** (Mobile) e del **Telemetria Ingestor** (Backend), testati tramite l'IoT Mock Simulator per validare il flusso di dati end-to-end in scenari di connettività intermittente.